

Présentation du sujet de programmation mission

1. Contexte

Ce BE a pour objectif de vous faire manipuler des algorithmes de mécanique spatiale via la bibliothèque PATRIUS (bibliothèque Java de dynamique du vol largement utilisée dans les outils CNES et développée par Thales) sur des problématiques de programmation mission. Les résultats seront directement visualisables avec l'outil VTS (outil CNES).

Une liste de villes françaises est fournie. Chaque site a un score associé à sa population. Nous utiliserons un seul satellite, décrit sur une base Pléiade avec une orbite héliosynchrone. Le but final du BE est de décrire le plan cinématique du satellite (loi d'attitude décrite sur tout l'horizon mission) pour que celui-ci puisse viser ces sites lors de ses différents passages. Pour y parvenir, il faudra au préalable calculer le plan d'accès pour chaque ville, puis le plan d'observation et enfin le plan cinématique du satellite. L'objectif adjoint à ce premier objectif est de maximiser le score total du plan en priorisant l'observation des villes les plus importantes ainsi qu'en minimisant l'angle d'incidence de visée de chaque observation.

Différentes contraintes mission sont à prendre en compte lors de la résolution.

Le sujet propose une méthode de résolution dans un contexte typique basique.

2. Présentation des outils

a) Eclipse

Le projet sera développé sous forme de code Java qui utilisera les fonctionnalités de la bibliothèque PATRIUS. On se basera sur Java 1.8.

Pour développer ce code on utilisera l'environnement de développement (IDE) libre d'accès Eclipse. Cet IDE présente toutes les fonctionnalités nécessaires au développement, à la compilation et l'exécution du projet.

b) PATRIUS

La bibliothèque du patrimoine de base du CNES PATRIUS fournit une brique de base algorithmique à de nombreux projets du CNES.

Cette bibliothèque contient les bases suivantes :

- Bases mathématiques,
- Bases de dynamique du vol,
- Orbitographie,



- Attitude,
- Véhicule,
- Mission.

Ce projet réutilisera largement certaines fonctionnalités de PATRIUS qui ne seront donc pas à recoder. Un Wiki et une Javadoc complète sont accessibles. On vous invite à vous en servir pour découvrir les différentes fonctionnalités disponibles. On vous met aussi à disposition une formation PATRIUS accélérée sous forme de séance de TD pour vous familiariser avec les fonctionnalités principales de PATRIUS. Vous pouvez aussi vous en inspirer pour vos développements.

Liens utiles :

- Wiki : <http://patrius.cnes.fr/index.php/Accueil>
- Javadoc : <http://patrius.cnes.fr/uploads/JavaDocs/V4.9.1/>

Au cours de ce BE, vous utiliserez la version PATRIUS 4.9.1 (incluse dans l'atelier de développement).

c) VTS

VTS (Visualisation Tool for Space data) est un outil de visualisation de données spatiales. Il permet de visualiser dans une simulation 3D un ou plusieurs satellites ainsi que leurs sous-systèmes (capteurs, panneaux, etc.) et de les faire évoluer dans un contexte spatio-temporel 3D affichant des corps célestes, des zones ou points d'intérêt, les trajectoires, l'emprise des capteurs, etc. VTS est basé sur le logiciel Celestia et partage donc ses contrôles et ses graphismes.

Le projet utilisera une version préconfigurée de VTS fournie avec le sujet. Dans cette version préconfigurée, nous fournissons un modèle 3D pour le satellite un projet VTS configuré sur mesure pour le projet. VTS lira en input ce contexte préconfiguré ainsi que les sorties des codes que nous vous proposons de développer, sachant que des outils d'écriture de ces fichiers de sortie vous seront également fournis. Ces fichiers correspondent en pratique à des éphémérides tabulées de trajectoire (position et vitesse) et d'attitude (quaternions) de votre satellite, aux événements orbitaux (plan cinématique) et aux zones d'intérêt (cibles d'observation) à observer.

*Remarque : Nous utiliserons VTS version 3.5.1 pour ce projet. Ce n'est pas la dernière version mais cette version est suffisante pour notre application en terme de fonctionnalités et performances. **Il faut noter que VTS n'est disponible que sur Windows et Linux.** Nous vous invitons à former vos groupes de manière à avoir au moins un PC Windows (nous ne fournissons que la version Windows de VTS pour éviter de trop alourdir le package du projet). Si vous travaillez sous Linux (ou utilisez une machine virtuelle Linux), il vous faudra l'installer par vous-mêmes et il y aura une petite modification à faire dans le code pour bien rediriger les outputs VTS vers le bon dossier. Pas d'inquiétude, tout est détaillé dans la section du sujet dédiée à l'installation de VTS Linux.*

Lien utile : site officiel VTS : <https://timeloop.fr/vts/>

3. Installation de l'environnement de développement

a) Prérequis 1 : Eclipse et Java

Si vous disposez déjà d'Eclipse (version assez récente si possible) avec Java 1.8, votre configuration actuelle devrait être suffisante et vous pouvez passer directement à la suite. Sinon, il faut installer l'environnement de développement Eclipse sur le PC de travail avec Java 1.8 :

1. Télécharger Eclipse ici : <https://www.eclipse.org/downloads/>
2. Une fois l'installateur lancé, sélectionnez « Eclipse IDE for Java Developers ».
3. Eclipse est maintenant installé. La version du JDK intégrée par défaut est normalement suffisante pour le projet. Vous pouvez passer à la suite.

b) Prérequis 2 : dossier « Environnement_de_developpement »

Récupérez le dossier "Environnement_de_developpement". Ce répertoire sera le répertoire parent contenant les codes et logiciels de votre projet. **Veillez à placer ce dossier à un emplacement dont le chemin n'est pas trop long/en dehors d'emplacements drive, sans quoi la procédure risque de ne pas fonctionner.**

Il contient :

Nom	Modifié le	Type	Taille
 BEProgrammationMission	15/09/2022 15:14	Dossier de fichiers	
 Vts-WindowsNT-32bits-3.5.1	15/09/2022 15:00	Dossier de fichiers	

- Le dossier « BEProgrammationMission » qui correspond au projet Eclipse que vous allez installer, contenant :

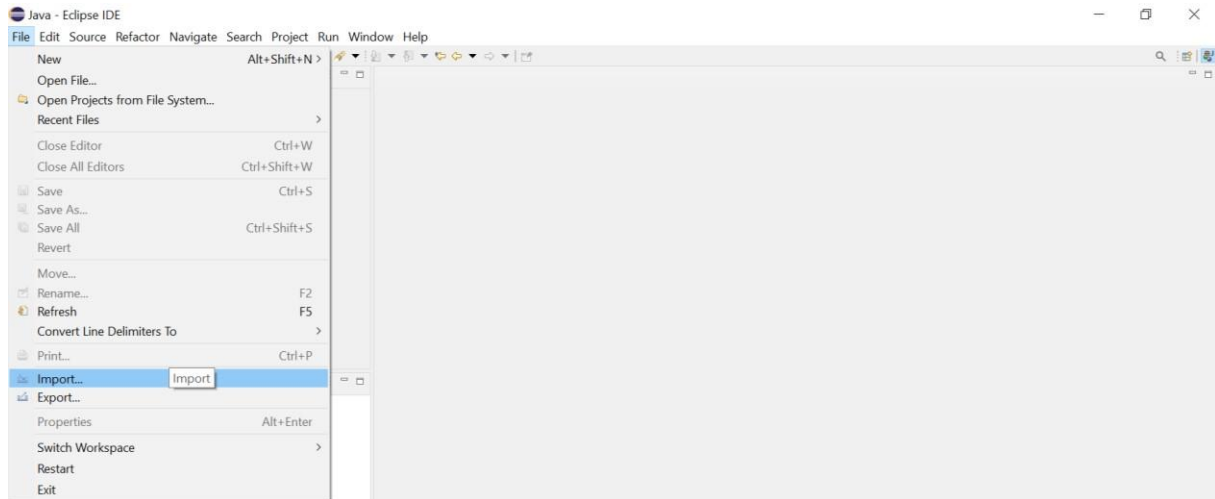
Nom	Modifié le	Type	Taille
 bin	15/09/2022 15:23	Dossier de fichiers	
 jars	12/09/2022 10:39	Dossier de fichiers	
 src	12/09/2022 09:30	Dossier de fichiers	
 .classpath	15/09/2022 15:16	Fichier CLASSPATH	3 Ko
 .project	12/09/2022 09:10	Fichier PROJECT	1 Ko

- Le dossier « src » qui contient lui-même :
 - Les codes sources fournis avec le sujet
 - Les ressources fournies avec les codes (liste des sites d'intérêt dans un csv)
- Le dossier « jars » contenant les dépendances à PATRIUS, au PatriusDataset ainsi que d'autres dépendances de plus bas niveau (ne pas modifier)
- Le dossier « bin » contenant tous les produits de compilation Java (ne pas modifier)
- Les fichiers « .classpath » et « .project » qui configurent votre projet Java (ne pas modifier)
- Des dossiers additionnels « logs » et « accesess », contiennent les logs et les accès sérialisés produit par le projet lors de son exécution

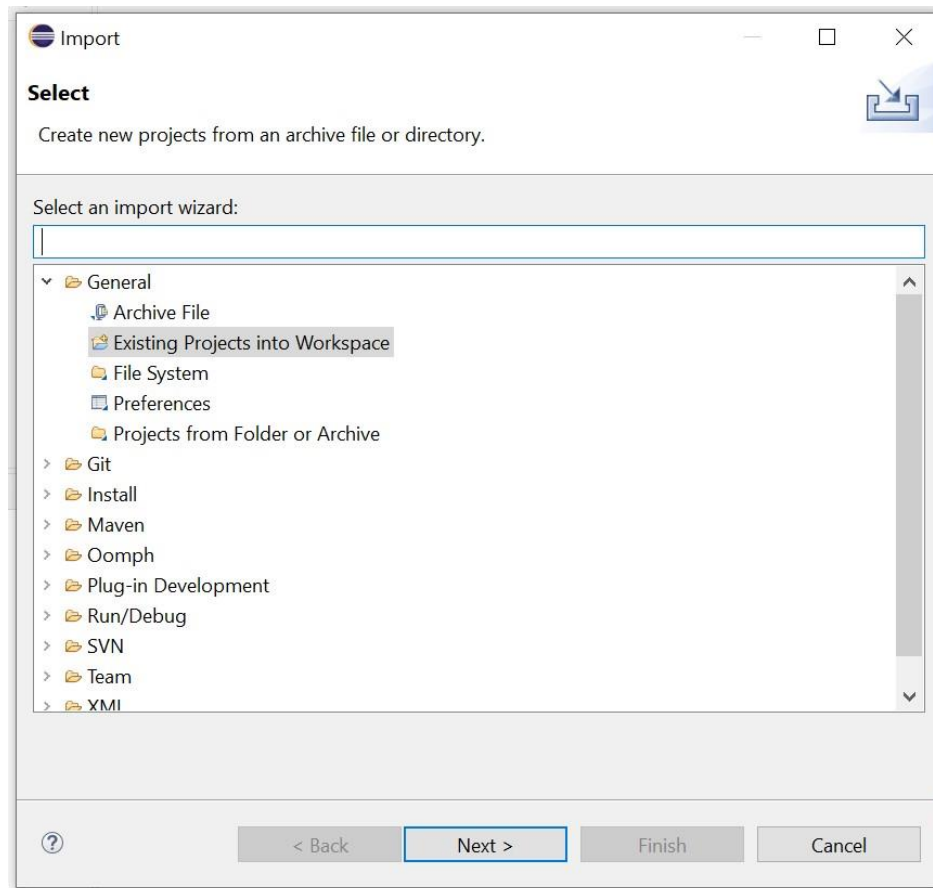
- Le dossier « Vts-WindowsNT-32bits-3.5.1 » qui contient le logiciel VTS et votre projet VTS dans le cadre du BE. Tout a été adapté au BE et normalement vous n'avez pas à modifier les fichiers contenus dans ce dossier. Voir le tuto VTS pour plus de détails là-dessus.

c) Création de votre projet Java

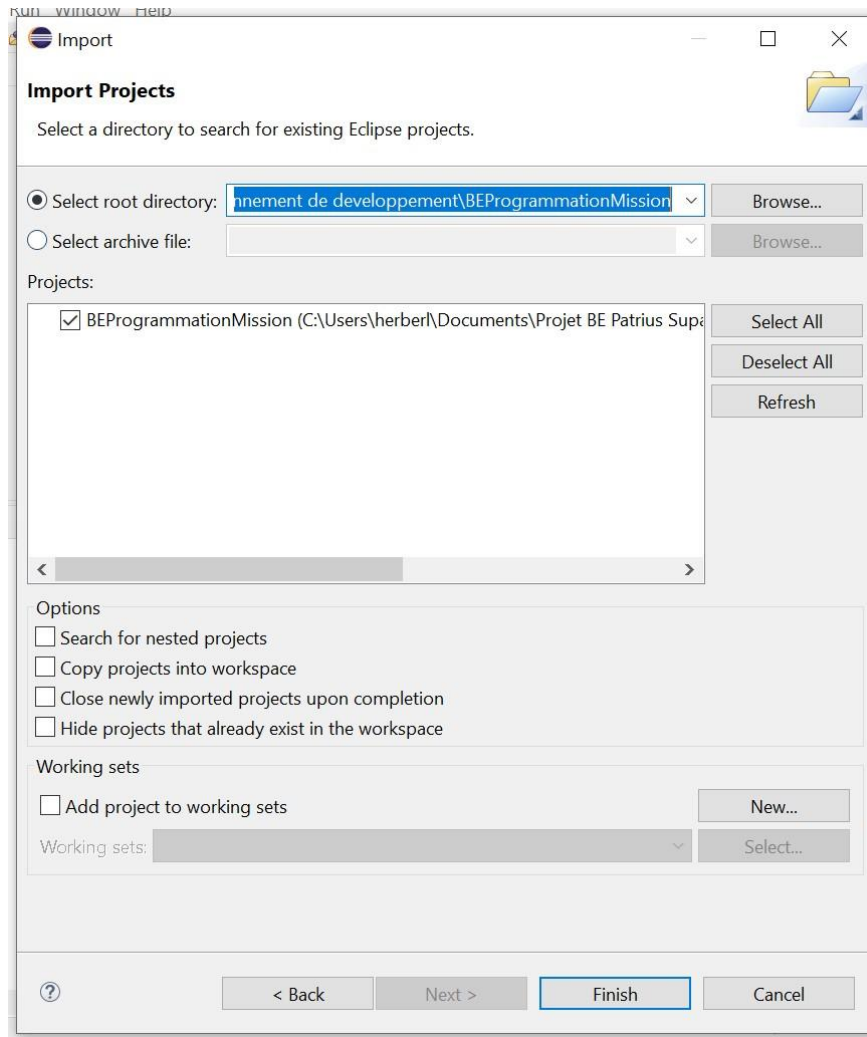
1. Ouvrir Eclipse
2. Puis cliquez sur : File > Import



3. Cliquez ensuite sur : General > Existing Projects Into Workspace



4. Vous devriez voir la fenêtre suivante, dans « Select root directory », renseignez le chemin vers le dossier fourni « BEProgrammationMission »



5. Dans « Projects », cochez la case « BEProgrammationMission » et cliquez sur Finish

Note : Dans la configuration du projet préparée ici, vous disposez en local de toutes les sources compilées des dépendances utilisées, vous permettant d'exécuter tous les codes sans encombre. Lors de vos développements, n'hésitez donc pas à CTRL+CLIC sur les classes et méthodes PATRIUS sur lesquelles vous souhaitez plus d'informations, Eclipse vous affichera directement le code source PATRIUS et la Javadoc associée. Cela vous donnera également des indices sur types des objets en entrées/sorties de chaque méthode et pourra vous orienter dans la résolution du BE.

Normalement, votre projet Eclipse est bien installé avec toutes les dépendances configurées. Vous pouvez démarrer le projet !

4. Résolution du sujet

a) Etape 1 : Calcul du plan d'accès

La première étape consiste à calculer quand le satellite peut accéder aux différents sites. Chaque accès sera une opportunité d'observation à considérer dans le calcul du plan d'observations qui suivra à l'étape 2 du BE.

Le plan d'accès sera calculé en prenant en compte ces différentes contraintes :

Contrainte de visibilité du satellite

La visibilité du satellite, soumise à une capacité maximale de dépointage, est modélisée par un cône qui décrit l'angle de dépointage maximal par rapport à l'axe nadir du satellite (verticale descendante, à l'opposé du zénith). Cela permet de s'assurer que, de manière générale, les sites inclus dans ce cône ne seront pas pointés avec des incidences de visée trop élevées (ce qui rendrait les acquisitions inexploitable).

Définition : l'angle d'incidence mentionné ici est l'angle entre la direction satellite-cible et la verticale locale de la cible (son zénith). Il est corrélé à l'angle de dépointage sans toutefois lui être égal !

Pour modéliser cette contrainte, nous mettons à votre disposition la valeur de l'angle du cône de dépointage maximal dans **ConstantsBE**, il s'agit du champ **POINTING_CAPACITY** que nous vous invitons à utiliser dans votre code.

Contrainte d'illumination des villes

Afin de contribuer à l'obtention d'images de bonne qualité, l'observation doit avoir lieu de jour dans de bonnes conditions d'illumination solaire. Pour cela, on analyse l'angle d'incidence solaire locale pour chaque site au moment de l'observation par le satellite. En dessous d'un angle minimum fixé dans les paramètres du projet, on considère que l'ensoleillement sur le site est suffisant pour que celui-ci soit imagé. On ne s'intéresse donc pas aux heures solaires locale pour chaque site mais uniquement à l'angle d'incidence solaire locale.

Définition : l'angle d'incidence solaire locale est l'angle entre la normale locale sortante au site d'intérêt (son zénith) et la direction site-soleil.

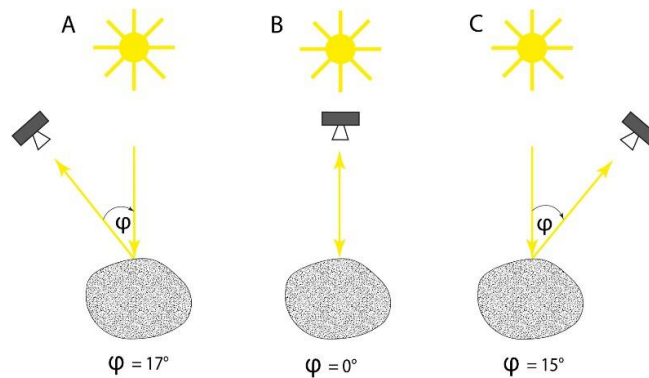
Pour modéliser cette contrainte, nous mettons à votre disposition la valeur de l'angle du cône d'incidence solaire maximale dans **ConstantsBE**, il s'agit du champ **MAX_SUN_INCIDENCE_ANGLE** que nous vous invitons à utiliser dans votre code.

Contrainte d'angle de phase solaire (non éblouissement direct)

Afin de protéger les optiques et éléments électroniques de la lumière directe du Soleil, on doit s'assurer qu'on n'est pas en éblouissement direct avec ce dernier. Exprimée géométriquement, cette contrainte est la suivante : l'angle de phase, à savoir l'angle entre le vecteur cible-satellite et le vecteur cible-

Soleil, doit être inférieur à 90° . Vous pouvez retrouver et utiliser cette valeur dans le code, grâce au champ **MAX_SUN_PHASE_ANGLE** de **ConstantsBE**. Si cette contrainte est bien respectée, le Soleil sera toujours hors du champ de vision du capteur car ses rayons seront au pire perpendiculaires à la direction de visée du capteur.

Ci-dessous une illustration venant du site de l'ESA (https://www.esa.int/ESA_Multimedia/Images/2019/12/Phase_angle_changes) :



Note : On ne s'intéresse pas à l'éblouissement par réflexion spéculaire (éblouissement issu des rayons réfléchis sur une surface plane).

Consignes de calcul

Pour cette première étape, on considère uniquement la trajectoire (orbite) du satellite, son cône de visibilité, la position du Soleil et la répartition des différents sites sur l'ellipsoïde de la Terre (points géodésiques : longitude, latitude, altitude). Il n'y a pas encore de notion d'attitude du satellite.

Le but sera de propager le satellite sur la fenêtre de mission [$\text{date}_{\text{initiale}}$; $\text{date}_{\text{finale}}$] et de détecter toutes les périodes qui remplissent chaque condition décrite précédemment, pour chaque site. Il faudra ensuite mettre en place un système de portes logiques pour extraire les intervalles de dates durant lesquels ces trois conditions d'observation sont vérifiées. On récupère ensuite les intervalles de dates qui correspondent au plan d'accès pour chaque site. Ce plan d'accès pour chaque site sera affiché en console grâce à la méthode **ProjectUtilities#printTimeline()**.

Les étapes de résolution sont directement décrites dans la Javadoc de la méthode **CompleteMission#computeAccessPlan()** qu'il faudra compléter, ainsi que celle des méthodes appelées.

Sérialisation des accès

Comme vous pourrez le voir dans le code, nous avons mis en place un système de sérialisation automatique des accès calculés. Sérialiser un objet Java (ou Python par exemple), c'est l'écrire en dur dans une interface (un fichier, lisible en clair ou non). Le calcul des accès prenant un certain temps, cela permet d'écrire les résultats dans des fichiers « .ser » contenant strictement le contenu de l'objet Java (l'objet Patrius **Timeline**), puis de simplement lire ces fichiers pour ensuite recharger les accès déjà

calculés et ne pas avoir à refaire le calcul. Ces fichiers sont tous stockés dans le dossier « *accesses* » du projet. Chaque fichier est nommé comme ceci : `NomDuSite_Hash.ser`, par exemple « `ALBI_562931739.ser` ». Le nom du site correspond au nom de la ville et le Hash correspond à un code unique généré à partir de toutes les constantes du BE et l'horizon mission, ce qui permet au moment du chargement des accès de vérifier que les accès qu'on charge pour un site donné ont bien été calculés avec les mêmes constantes et le même horizon mission (par comparaison des codes Hash), sans quoi on doit recalculer l'accès.

Si vous souhaitez recalculer les accès d'un site, il vous faudra donc effacer les fichiers « `.ser` » du site concerné. Vous pouvez aussi modifier directement le code pour désactiver la sérialisation et sa lecture lorsque vous développez, puis la réactiver une fois que vous avez validé que votre calcul d'accès est correct (avec VTS).

Exécution du programme

Une fois le calcul de plan d'accès finalisé, la classe ***CompleteMissionMain*** peut être exécutée en commentant le calcul du plan d'observation et le calcul du plan cinématique (étape 2 du BE). L'exécution du calcul du plan d'accès doit permettre d'afficher les ***Timeline*** d'accès de chaque site dans la console.

Evaluation des résultats avec VTS

La classe ***SimpleMission*** fournie dans le projet Java permet de générer les fichiers VTS pour observer le passage du satellite au-dessus des stations avec une loi d'attitude nadir. Pour générer ces fichiers il faut exécuter la méthode ***SimpleMission#createSimpleVTSVisualization()***. Cette méthode est disponible depuis la classe ***CompleteMission*** puisque celle-ci étend la classe ***SimpleMission*** dans laquelle la méthode est définie. On peut ensuite lancer VTS et visualiser le satellite et les sites d'intérêt sur l'horizon mission.

On vérifiera les résultats du plan d'accès affiché dans la console par cohérence visuelle avec VTS. En utilisant la simulation VTS simple qui permet de visualiser la position du satellite sur toute la fenêtre mission avec son cône de visibilité et les passages jour / nuit, on doit pouvoir vérifier manuellement pour un site donné si les conditions de visibilité, d'illumination et d'angle de phase sont bien respectées au cours des intervalles temporels contenus par le plan d'accès obtenu.

Remarque au sujet du code : dans le constructeur de ***SimpleMission***, la variable *numberOfSites* permet de sélectionner le nombre de sites à considérer pour l'analyse. Cette variable est comprise entre 1 et le nombre total de communes en France. Dans le cadre du développement, il est conseillé de ne pas considérer tous les sites pour accélérer l'exécution du programme. Lors des évaluations, on considérera un nombre constant de sites commun à tous les groupes.

b) Etape 2 : Calcul du plan d'observation et du plan cinématique

Maintenant que le plan d'accès est calculé, les prochaines étapes vont être de calculer le plan d'observations et le plan cinématique.

Comme pour le calcul du plan d'accès, les étapes de résolution sont directement décrites dans la Javadoc des méthodes **`CompleteMission#computeObservationPlan()`** et **`CompleteMission#computeCinematicPlan()`** qu'il faudra compléter, ainsi que les méthodes appelées. Certains modules de la formation PATRIUS sont directement identifiés et pourront vous être utiles.

Pour ces deux dernières étapes, nous vous proposons une approche avec plusieurs niveaux de résolution pour prendre en main progressivement le code, le but étant d'atteindre le niveau le plus haut.

Nous recommandons de compléter le code des deux étapes (calcul plan d'observation et calcul du plan cinématique) pour un niveau donné avant d'essayer le niveau suivant car ces deux étapes sont fortement liées.

i. Calcul du plan d'observation

Le plan d'observations consiste à décrire la stratégie d'ordonnancement des observations à réaliser. Plus précisément, il va s'agir de construire un enchainement d'observations sur l'horizon mission :

- Qui se base sur les créneaux d'accès précédemment calculés pour chaque ville
- Qui ne peut observer qu'une seule ville à la fois
- Qui ne peut observer une ville qu'une seule fois maximum sur l'horizon mission
- Qui soit réaliste cinématiquement (respect des contraintes dynamiques du satellite)
- Qui maximise le score total : somme des scores de chaque ville observée pondéré par le cosinus de l'angle d'incidence de l'observation.

Loi de guidage d'observation

Observer un site consiste à le **viser pendant 10 secondes avant de pouvoir dépointer**. C'est donc ainsi que nous définissons dans le cadre de ce projet la notion de **guidage d'observation**.

Pour modéliser cette contrainte, nous mettons à votre disposition la valeur du temps de pointage/observation constant dans **`ConstantsBE`**, il s'agit du champ **`INTEGRATION_TIME`** que nous vous invitons à utiliser dans votre code.

Note : nous prenons ici cette simplification car elle est suffisante pour établir un plan d'observation réaliste. En réalité, il faudrait développer un guidage d'observation plus réaliste pour un capteur push-broom type Pléiades, mais cela prendrait beaucoup plus de temps et ce n'est pas pertinent d'aborder cette modélisation dans le cadre de ce BE.

Calcul du score en fonction de l'incidence d'observation

De plus, pour évaluer le score d'une observation donnée, on calculera l'angle d'incidence d'observation (ou incidence de visée) à la date du milieu de l'observation. Cela permet en effet de modéliser de manière grossière un critère simple de qualité image pour l'observation : plus l'angle d'incidence est

faible, plus l'observation sera de bonne qualité (géométriquement). A l'inverse, plus l'angle est élevé, plus l'image sera déformée.

Cet angle est l'angle formé entre le vecteur normal à la surface de la Terre sortant du site ciblé (son zénith) et le vecteur cible-satellite à la date du milieu d'observation.

Le cosinus de cet angle est utilisé pour prendre en compte le facteur géométrique d'incidence. On aura donc une contribution de l'observation qui sera égale à :

$$\text{score_ville_effectif} = \cosinus(\text{incidence_observation}) \times \text{score_ville}$$

Les contributions de chaque ville seront sommées afin d'obtenir le score total du plan.

Il faudra donc veiller à bien positionner temporellement l'observation de chaque ville (durant un des intervalles d'accès à cette ville) pour maximiser la contribution de chacune au score total. Chaque ville n'étant insérable qu'une seule fois dans le plan d'observation, il faudra trouver un équilibre entre observer une ville avec une incidence proche de zéro (quitte à ne pas pouvoir observer certaines autres villes), ou bien dégrader légèrement la qualité des observations pour pouvoir observer plus de villes.

Note : Quoiqu'il arrive, observer un site respectant les contraintes d'accès permettra d'obtenir un angle d'incidence borné qui vous assurera que le cosinus de cet angle ne vaut pas zéro et donc que l'observation vous rapporte des points, mais optimiser l'incidence des observations fera clairement la différence lors du calcul du score total, surtout pour les villes à gros score (Paris, Marseille, etc.).

Contrainte cinématique

En plus de la contrainte d'observation, nous ajoutons une contrainte générale sur la cinématique du satellite. Elle doit être respectée quel que soit l'instant de l'horizon mission, notamment entre les observations. La contrainte est la suivante : **les capacités d'agilité du satellite (type Pléiades) doivent être respectées dans le plan d'observation**. C'est-à-dire que le satellite peut effectuer un angle donné de dépointage en un temps donné, tous deux contraints par une loi.

Cette loi est fournie dans le code du BE sous forme d'une méthode : **Satellite#computeSlewDuration()** qui vous permet de calculer la durée de ralliement minimum en fonction de l'angle visé ou d'un couple d'attitudes (attitude initiale et finale). Cette méthode est basée sur une interpolation polynomiale par splines d'une table de valeurs « angle à rallier » vs « durée de ralliement ». Ces valeurs sont décrites dans la Javadoc de **ConstantsBE** dans les champs **POINTING_AGILITY_ROTATIONS** et **POINTING_AGILITY_DURATIONS**.

Il faudra donc **veiller à ce que les durées qui séparent les différents modes de pointage (nadir ou observation d'un site) définis par votre enchaînement d'observations soient toujours plus longues que la durée minimale théorique calculée par la méthode**. Cela suffira à démontrer que le satellite ne dépasse jamais ses capacités d'agilité et est donc capable d'accomplir le plan ainsi proposé.

Note : Cette contrainte est automatiquement respectée au cours d'une observation, malgré le défilement, car les capacités d'agilité de Pléiades sont dimensionnées pour s'adapter (très largement) à la cinématique de visée d'un point fixe au sol en orbite héliosynchrone.

Niveaux de résolution

Plusieurs niveaux de résolution classés par niveau de difficulté sont proposés pour calculer le plan d'observation. Ces niveaux sont donnés à titre indicatif pour vous permettre de progresser pas à pas, le but étant d'implémenter le niveau 3.

En sortie de simulation, vous devrez afficher dans la console la liste des observations (sites visés et intervalles de dates d'observation associés) ainsi que le score total obtenu qui s'obtient via la méthode **`CompleteMissionSolution#computeFinalScore()`**.

Niveau 1

La stratégie la plus simple pour remplir le plan d'observations est de sélectionner manuellement une seule ville (différente à chaque fois) par passage.

Exemple : Si Strasbourg est visible au 1^{er} passage, on observe cette ville, si Paris est visible au 2^e passage on observe cette ville, etc... sur tout l'horizon mission.

Cette stratégie permet de s'affranchir des contraintes cinématiques et peut être réalisée « manuellement » sans algorithme de remplissage automatisé du plan d'observations. Cette stratégie est évidemment sous-optimale puisque très peu de sites seront observés sur tout l'horizon mission, mais elle permet de commencer à se familiariser avec les différentes étapes du calcul.

Niveau 2

Cette stratégie consiste à considérer une durée de ralliement constante à respecter entre deux guidages. Cette durée correspond au temps nécessaire pour effectuer un dépointage « pire cas » théorique.

Exemple : On calcule la durée de dépointage pour un angle de deux fois l'angle de dépointage maximal autorisé (pour rappel cet angle prend en compte la dynamique du satellite). Cela revient à parcourir le cône de visée d'un bord à l'autre dans une direction donnée. On s'assure ainsi que le satellite aura le temps (cinématiquement parlant) de passer d'un site à un autre avec cette durée minimale, car les deux sites se situent nécessairement dans ce cône de visée au moment des observations. On peut donc remplir le plan d'observation sans risque lorsqu'on considère ce type de ralliement à durée fixe.

Pour cette stratégie, vous pouvez utiliser la méthode **`Satellite#getMaxSlewDuration()`** qui vous donne automatiquement la durée associée à ce dépointage « pire cas » à durée maximale.

Cette stratégie permet de simplifier les contraintes cinématiques à considérer avec une seule loi de durée constante à appliquer. Ce niveau permet d'observer plusieurs villes sur un même passage mais il reste sous-optimal car le passage d'une ville à l'autre se fera plus lentement que ce que le satellite est capable de faire (capacités dynamiques sous-exploitées).

Niveau 3

Note : il est fortement déconseillé de commencer le niveau 3 si vous n'avez pas réussi à produire un plan cohérent avec une stratégie de niveau 1 ou 2, plan cinématique inclus !

Cette stratégie considère toute la dynamique du système et propose de traiter le problème de remplissage du plan d'observations comme un problème d'optimisation complet, avec pour but de maximiser le score total.

Pour respecter la dynamique du système, il faut calculer le temps que met le satellite à dépointer d'un site A vers un site B en fonction de ses contraintes cinématiques. On calcule ainsi la durée nécessaire au satellite pour faire le ralliement entre la fin de l'observation de A et le début l'observation de B.

Cette fois, vous aurez besoin de la méthode **Satellite#computeSlewDuration(Attitude 1, Attitude 2)**. Puisque cette méthode prend en entrée des objets de type **Attitude**, vous aurez donc également besoin de calculer la loi d'attitude de pointage des différents sites afin de pouvoir récupérer l'attitude du satellite à n'importe quelle date de l'horizon de simulation.

Pour cela, jetez un coup d'œil à la méthode **CompleteMission#createObservationLaw()** : celle-ci doit instancier et exploiter les caractéristiques de la loi de guidage **TargetGroundingPointing** pour chaque site que l'on souhaite observer. Créer les lois dès l'étape du plan d'observation vous permettra d'anticiper le calcul du plan cinématique.

Comme pour le niveau 2, pour résoudre ce problème d'optimisation, vous devrez proposer un algorithme capable de définir l'ordre d'observation des sites qui respecte ce critère cinématique (mais cette fois la durée de ralliement cinématique dépend des villes observées).

Exemple : « algorithme greedy » présenté en amphithéâtre lors de la première séance, voir les slides associées, nous les mettrons en ligne après la première séance.

Vous devez choisir votre propre stratégie de choix des sites à observer et de leur ordonnancement en essayant de maximiser le score total.

ii. Calcul du plan cinématique

Maintenant que l'on connaît toutes les observations qui auront lieu sur l'horizon mission avec la certitude que ces observations et leurs transitions sont cinématiquement réalisables, il reste alors à calculer la cinématique du satellite à chaque instant de l'horizon mission, afin de suivre le plan d'observation défini précédemment. La cinématique du satellite sera visualisable directement dans VTS.

Comme pour le calcul du plan d'observation, on propose plusieurs niveaux de difficulté pour avancer pas à pas dans la résolution de cette étape, le but est d'arriver au niveau 2.

Niveau 1

Suivre les étapes suivantes :

- Utiliser la méthode **CompleteMission#createObservationLaw()** qui permet d'instancier un **TargetGroundingPointing** pour chaque site que l'on souhaite observer.
- Construire les **AttitudeLawLeg(TargetGroundPointing pointing, AbsoluteDate dateStart, AbsoluteDate dateEnd, String legName)** correspondant aux observations. Ne pas oublier de nommer chaque leg différemment (en utilisant le nom de la ville visée par exemple) pour ne pas risquer de les mélanger a posteriori.
- Construire une **StrictAttitudeLegsSequence** qui décrit la loi d'attitude du satellite sur l'horizon mission (cet objet représentera le plan cinématique) :
 - Loi de visée nadir au début
 - Ralliement loi nadir -> guidage observation 1
 - Guidage observation 1
 - Ralliement vers la 2° observation
 - Guidage observation 2
 - Ralliements, observations, etc...
 - Guidage observation N (dernière observation)
 - Ralliement guidage observation N -> loi nadir
 - Loi de visée nadir jusqu'à la fin
- Utiliser **CompleteMission#generateVTSTVisualization()** pour écrire le fichier cinématique VTS

Niveau 2

La complexité suivante est ajoutée : le satellite doit repasser en guidage nadir entre deux observations si la transition d'une ville à l'autre est suffisamment longue pour le permettre.

On peut en déduire la conséquence suivante : entre les observations N et N+1, si la durée du ralliement est supérieure à deux fois la durée maximale de ralliement (donnée par le getter **Satellite#getMaxSlewDuration()**), alors plutôt que d'insérer uniquement un grand ralliement de l'observation N à l'observation N+1, on sera également en mesure d'insérer un nadir transitoire. La séquence sera alors la suivante : observation_N => nadir => observation_{N+1}.

La durée du nadir sera égale à :

$\text{durée_guidage_nadir} = \text{durée_totale_transition} - 2 \times \text{durée_maximale_ralliement}$

Note : repasser en guidage nadir à chaque fois qu'on a le temps permet d'éviter d'être ébloui par le Soleil de manière directe lors des transitions. Par exemple pour Pléiades 1A, entre une passe descendante au-dessus de la France exploitable et la suivante, on a toujours largement plus de temps que la durée de ralliement. Ainsi, si on ne contrôle pas l'attitude du satellite pendant toute cette durée disponible (équivalente à plusieurs orbites), le satellite peut alors se retrouver à pointer en direction du Soleil. Dans le cadre du BE on ne repasse en nadir que quand on a du temps libre car le satellite pointe déjà le sol entre deux observations proches, il y a donc de très faibles risques qu'il se retrouve à un moment ébloui. La réflexion spéculaire est un autre critère qu'il faudrait gérer, mais celle-ci étant complexe, elle est hors du cadre de ce BE.

iii. Sorties

En sortie du calcul du plan cinématique, il faut pouvoir afficher la **StrictAttitudeLegsSequence** dans la console ainsi que les informations des sites visés. En pratique, vous pourrez utiliser la méthode **StrictAttitudeLegsSequence#toPrettyString()** pour afficher votre plan cinématique. Cette méthode vous affichera la séquence de legs dans l'ordre chronologique, avec l'intervalle temporel et le nom de chacun. Il est donc important de bien nommer tous vos legs à l'étape de calcul du plan d'observation.

Une fois le plan cinématique construit, la méthode **CompleteMission#checkCinematicPlan()** permettra de vérifier automatiquement que la durée de chaque leg est bien supérieure à la durée théorique que mettrait le satellite à effectuer un ralliement entre les attitudes en début en fin de ce leg (que ce soit un leg de ralliement ou non). Cette condition sert avant tout à vérifier la cinématique des ralliements, puisque les guidages nadir et **TargetGroundPointing** ne sont pas exigeants dans notre contexte de simulation grâce aux contraintes fixées par le plan d'accès. Ces deux guidages ne devraient donc jamais lever d'erreur dans le plan cinématique.

Si le plan cinématique est validé, vous pourrez alors faire un appel à la méthode de calcul des scores : **CompleteMission#computeFinalScore()**.

Enfin, vous pourrez appeler la méthode **CompleteMission#generateVTSVisualization(cinematicPlan)** pour générer tous les fichiers nécessaires à une visualisation complète avec VTS (position et attitude du satellite sur l'horizon mission).

Lors des différentes séances, nous vérifierons visuellement la cohérence du déroulement de la mission dans VTS.

c) Bonus : implémentation de tests unitaires

i. Contexte : les différents types de tests

Lors de la production de code industriel, la validation du code est une étape primordiale, aussi importante que le développement. Il n'est d'ailleurs pas rare que la validation prenne plus de temps que le développement en lui-même. Cette étape permet de vérifier le bon fonctionnement de toutes les méthodes en partant des plus basiques jusqu'aux méthodes haut niveau. Cela peut se faire avec différents types de tests :

- Les tests unitaires : ces tests portent sur les composants de bas niveau du code. On vérifie que chaque composant du code peut être exécuté avec succès, c'est-à-dire que le code s'exécute sans renvoyer d'erreur dans un cas nominal très basique et que les résultats sont conformes à des résultats attendus. On peut également vérifier que la bonne exception est traitée et renvoyée dans un cas corrompu. On se contente ici de cas d'exécution basiques avec des résultats évidents, on ne va pas chercher à valider en détail le composant testé. Ces tests sont par conception automatisables afin d'être joués à chaque modification du code.
- Les tests de non régression : ces tests peuvent porter sur des composants bas niveau du code et plus rarement sur des fonctions complexes haut niveau car ils sont lourds à mettre en place. Le principe est le suivant : on code en dur un jeu de résultats attendus (qui ont été validés par

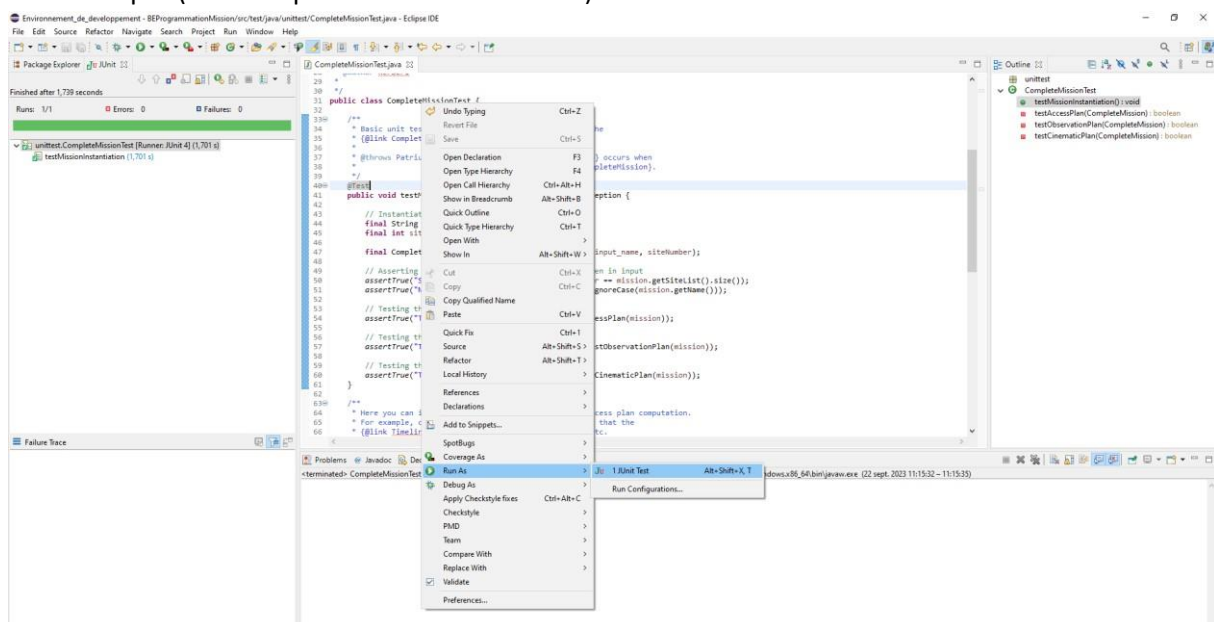
ailleurs) dans un cas d'exécution précis prenant en entrée un jeu de données figé. Ces tests peuvent être basiques ou complexes et on vérifie alors que, dans les mêmes conditions, le code renvoie toujours les mêmes résultats. Le but est de pouvoir facilement détecter une régression du code lors d'une montée de version car on voit tout de suite en jouant ces tests si un test ne renvoie plus les bons résultats, ce qui permet de détecter instantanément la partie du code affectée et de la corriger. Ces tests sont par conception automatisables afin d'être joués à chaque modification du code.

- Les tests fonctionnels sont quant à eux des tests plus haut niveau qui permettent de valider en détail le comportement des codes dans des situations réalistes. On va donc utiliser le code avec des jeux de données complexes dans un cadre réaliste et vérifier qu'il répond bien aux attentes. Ces tests sont souvent effectués lors de campagnes de validation jouées manuellement.

ii. Utilisation de l'outil JUnit

Pour implémenter les tests de manière automatisée, on utilise dans l'industrie des outils de test qui dépendent du langage : pytest pour Python par exemple, et pour Java on utilise parfois JUnit. C'est l'outil que nous avons choisi d'utiliser dans le cadre du BE.

Nous avons donc installé cet outil dans le projet via les dépendances Java (jars). De plus, nous fournissons une classe **CompleteMissionTest** dans le package `src/test/java/unitest` qui vous explique comment intégrer des tests unitaires en la complétant. Pour exécuter ces tests, il vous suffit d'exécuter cette classe la même manière qu'une classe « main » dans Eclipse en faisant un clic droit sur la classe à exécuter puis « Run As > JUnit Test ». Cela déclenchera l'exécution des tests avec JUnit, la mécanique n'est pas la même que l'exécution d'une classe « main » mais cela sera transparent pour vous. Vous verrez dans votre IDE Eclipse un volet « JUnit » s'ouvrir qui vous indique si vos tests sont effectués avec succès ou pas (voir la capture d'écran ci-dessous).



Quelques remarques :

- Les classes de test doivent par convention avoir un nom qui finit par « Test »
- Les méthodes des classes de test qui doivent être exécutées par JUnit lors de l'exécution de la classe de test doivent porter un balise « @Test » comme celle illustrée dans la capture d'écran ci-dessus. Cela permet à JUnit de les repérer, de les traiter et de les exécuter automatiquement.

iii. Consignes pour le BE

Dans le cas du projet, nous ne vous demanderons pas d'implémenter de tests de non régression ni de tests fonctionnels car ceux-ci sont plus complexes et demanderaient que vous ayez un moyen annexe de générer des résultats validés. En revanche, nous allons évaluer votre capacité à implémenter des tests unitaires simples pour vérifier de manière automatique que votre code fonctionne bien.

Pour cela, nous utiliserons l'outil JUnit installé avec le projet, qui vous sera présenté lors des séances. Il vous faudra compléter la classe **CompleteMissionTest** du package `src/test/java/unittest` afin d'y implémenter des tests unitaires pour valider le calcul du plan d'accès, le calcul du plan d'observation et le calcul du plan cinématique. Cette classe est commentée de manière à vous donner des exemples de tests unitaires effectués sur l'instanciation de l'objet **CompleteMission** et à vous guider en vous fournissant une structure de code à compléter et des consignes et astuces dans les commentaires du code directement.

Ces tests unitaires vous permettront de gagner quelques points bonus sur votre note finale : cela fait partie de l'évaluation de la qualité de votre code. Pour autant, vous n'êtes pas pénalisés si vous ne complétez pas la classe **CompleteMissionTest**.

5. Contexte et consignes pour le développement

a) Classes fournies

Voici les classes fournies et leur description :

- Dans le dossier source « src/main/java » :
 - Package « progmission » : contient les classes liées à la thématique de la programmation mission, ce sont donc toutes les classes qui servent véritablement à modéliser et résoudre le problème
 - **SimpleMission** : cette classe modélise une simple mission d'observation de la Terre où le satellite ne fait que pointer la direction nadir. Elle permet de définir le contexte physique du problème avec des objets PATRIUS : échelle de temps TAI, référentiel inertiel EME2000, corps céleste Earth pour la Terre, représenté par un ellipsoïde, modèle du Soleil, dates de début et de fin de la mission. Elle encapsule aussi l'objet Satellite que nous décrivons plus loin et qui modélise le satellite, ainsi que la liste des sites d'intérêts définis par leur nom et leurs coordonnées à la surface de l'objet Terre. Elle définit également des méthodes utiles commentées en Javadoc.
 - **Satellite** : la classe satellite modélise le satellite Pléiades 1A : sa géométrie (repère local attaché, différentes pièces attachées), sa masse, son agilité

(capacité de dépointage et calcul des ralliements), son capteur (axe de visée, modèle de capteur avec un objet PATRIUS). Elle définit aussi sa dynamique orbitale représentée par un propagateur d'orbite képlérienne réglé sur l'orbite de Pléiades 1A et sa loi d'attitude réglée par défaut sur la visée nadir.

- **CompleteMission** : cette classe est la seule que vous devez éditer de tout le dossier src/main/java pour résoudre le sujet. Il s'agit d'une classe qui étend **SimpleMission** (au sens Java) afin d'hériter du contexte physique et des méthodes qu'elle développe. L'idée est ensuite de modéliser dans cette classe votre plan d'accès aux sites d'intérêt, votre plan d'observation et votre plan cinématique. Pour cela, nous fournissons dans cette classe un certain nombre de méthodes à compléter avec une Javadoc détaillant les étapes à suivre et les objet PATRIUS à utiliser pour calculer vos plans. On fournit également un certain nombre de méthodes utilitaires (voir code).
- Package « reader » : ce package définit les classes utilitaires pour lire et modéliser les sites d'intérêt
 - **Site** : c'est la classe qui modélise un site d'intérêt. Elle permet d'y encapsuler son nom, son score pour le calcul du score de plan d'observation et ses coordonnées géographiques à la surface de l'objet Terre (Earth). Notez que pour modéliser un **Site** on utilise la classe PATRIUS **GeodeticPoint** qui encapsule ses coordonnées (longitude, latitude, altitude).
 - **SitesReader** : c'est le lecteur des sites d'intérêt. Le fichier d'input est un fichier CSV qui se trouve dans le dossier « src/test/resources ».
- Package « utils » : ce package définit toutes les classes utilitaires fournies pour vous simplifier la vie
 - **ConstantsBE** : cette classe définit toutes les constantes (ou presque) utilisées dans le BE. La Javadoc indique la signification et l'unité de chacune. Vous serez amenés à les utiliser pour résoudre le BE.
 - **ProjectUtils** : cette classe vous fournit une méthode simple permettant d'afficher les résultats de votre code de manière ergonomique.
 - **VTSTools** : cette classe fournit toutes les méthodes d'écriture des fichiers VTS qui permettent d'interfacer votre projet Java avec le logiciel VTS.
 - **LogUtils** : cette classe vous fournit un objet **Logger** que vous devrez utiliser pour loguer vos erreurs, informations, résultats, etc...
- Dans le dossier « src/test/java » :
 - Package « simulation » : ce package contient les classes « main » permettant d'exécuter les classes du package « progmission » et ainsi de simuler votre mission. Il s'agit des seules classes que vous allez pouvoir exécuter et qui produiront des sorties en console et des outputs VTS.
 - **SimpleMissionMain** : c'est le « main » pour la classe **SimpleMission** avec un code pré-rempli permettant de lancer une simulation de **SimpleMission** (voir le détail dans le code et ses commentaires). Cette classe vous servira d'exemple de démonstration pour vérifier que votre projet est bien installé et pour comprendre les notions basiques de propagation d'orbite avec une loi nadir ainsi que le contexte PATRIUS de la simulation.

- **CompleteMissionMain** : c'est le « main » pour la classe **CompleteMission**, avec un code pré-rempli qui effectue toutes les étapes listées dans le présent sujet. Il exécute à la suite toutes les méthodes de **CompleteMission** jusqu'à l'écriture des fichiers VTS.
- Package « tuto » : ce package contient la classe de tutoriel Patrius
 - **PatriusTuto** : classe de tutoriel utilisée lors de la séance de tutoriel Patrius, il s'agit d'un code que vous complèterez en séance pour vous familiariser avec les principales fonctions de Patrius utiles à la résolution du BE.
- Package « unittest » : ce package est dédié aux tests unitaires
 - **CompleteMissionTest** : c'est votre classe de tests, à éditer également (bonus). C'est ici que vous allez implémenter des tests unitaires basés sur l'outil JUnit qui est installé dans votre projet. Le code fourni est commenté pour vous donner des exemples de tests unitaires et vous guider dans l'implémentation de vos propres tests.

b) Consignes de développement

i. Dans quels fichiers pouvez-vous développer ?

- Dans le package « *src/main/java* » :
 - Pour résoudre le sujet du BE, **vous ne devez modifier que les méthodes de la classe *CompleteMission* pour lesquelles il y a un bandeau indiquant explicitement de « modifier cette méthode »**. Les autres méthodes ne doivent pas être modifiées et surtout pas supprimées.
 - Vous êtes cependant libres d'ajouter autant de méthodes que vous le souhaitez à la classe **CompleteMission**, ou encore lui créer des classes internes (dans le même fichier .java). **Ne créez cependant pas de contenu à l'extérieur de *CompleteMission***.
- Dans le package « *src/test/java* » :
 - Vous pouvez compléter la classe **CompleteMissionTest** pour obtenir des points bonus sur la qualité du code.
 - Vous pouvez vous créer des classes de type « main » (pour tester l'exécution de votre code), mais ces nouvelles classes ne feront pas partie du contenu que nous évaluerons. Toutes ces classes de type « main » ou « test » doivent être placées dans le package « *src/test/java* », c'est en effet la norme dans les projets Java industriels basés sur une architecture Maven. Cette pratique est largement utilisée dans l'industrie et notamment au CNES.

ii. Consignes qualité

Voici quelques bonnes pratiques qualité que nous vous demandons, dans la mesure du raisonnable, de respecter :

- **Noms des variables/classes/méthodes** : donnez des noms à la fois courts et clairs, les plus cohérents possibles. Un minimum de trois caractères est demandé pour les différents noms. Une tolérance est faite par exemple avec *x/y/z* pour des coordonnées. Dans un cas plus général, vous pouvez utiliser *satellite* ou *sat* pour un objet **Satellite**.
Les noms de classes, variables et méthodes doivent respecter un format camel case Java. Par exemple pour « angle provider » on écrira `AngleProvider` pour une classe et `angleProvider` pour une variable. On n'utilise pas de majuscule pour la première lettre des méthodes : on écrira donc `getCoordinates()` et jamais `GetCoordinates()`. L'utilisation de l'underscore est à proscrire également en dehors des variables statiques finales (représentent des constantes).
- **Commentez votre code** : expliquez ce que vous faites de manière à nous rendre la vie facile pour comprendre votre raisonnement et le fonctionnement de votre code. Il en va de même pour les éventuels tests unitaires que vous allez implémenter.
- **Factorisez votre code** : évitez de dupliquer des lignes de code. Si deux méthodes ont 5 lignes identiques, vous pouvez factoriser ces 5 lignes de code dans une même méthode appelée par vos deux méthodes, cela évite les erreurs de copier-coller et rend le code plus simple à lire, à comprendre et à maintenir.
De la même manière, si vous utilisez plusieurs fois le même **double**, la même **String**, le même **int** dans le code, déclarez sa valeur comme une constante une seule fois (en haut d'une classe par exemple) puis utilisez uniquement cette constante au lieu de réécrire sa valeur à chaque fois. Cela permet d'éviter les erreurs de copier-coller et si vous devez changer la valeur vous n'aurez qu'une instance à modifier au lieu de devoir modifier plusieurs instances partout dans le code.
- **Logging** :
Toutes les sorties « console » (autrement appelés les « logs ») doivent obligatoirement passer par le **Logger** mis à votre disposition. **Explication** :
Le logging, ou journalisation, correspond aux sorties visuelles du code pendant l'exécution, souvent via la console en faisant un `print` (`System.out.println()` en Java, `print()` en Python, etc...). Lors de vos développements précédents vous utilisiez peut-être simplement la console pour afficher vos sorties, mais cela présente l'inconvénient de perdre les logs une fois la console fermée, et le simple affichage console ne dispose d'aucune fonctionnalité supplémentaire. Dans l'industrie, on veut pouvoir garder les résultats d'un code pendant parfois plusieurs années, avec un niveau de verbosité (importance des logs : `DEBUG`, `INFO`, `WARNING`, `ERROR` par exemple) différent suivant les cas. Cela implique d'écrire les logs dans la console mais aussi dans des fichiers, parfois dans plusieurs fichiers différents avec des niveaux de verbosité paramétrables. Pour cela, on ne recode bien sûr pas tout, on utilise ce qu'on appelle un framework de logging, à savoir une dépendance qui gère les logs et nous fournit une classe **Logger** prête à être utilisée et paramétrable facilement. Dans notre projet c'est la dépendance `SLF4J` + `Logback-classic` qui gère les logs et nous vous avons préparé la classe **LogUtils** qui déclare de manière statique un unique **Logger** pour tout le projet appelé **LOGGER**. Nous vous imposons donc d'utiliser exclusivement ce **Logger** pour tout affichage dans la console.
En résumé, remplacez tous vos « `System.out.println(message)` » par « `LOGGER.info(message)` » ou `LOGGER.debug/warn/error(message)` suivant le degré de verbosité souhaité.

Remarques :

- Tous les logs seront aussi écrits dans un fichier de log nommé « logfile.log » situé dans le répertoire « logs » du projet.
- Notez aussi que la configuration des logs se fait via le fichier « src/test/resources/logback-test.xml » que nous vous demandons donc de ne pas modifier.

6. Utilisation et configuration de l'outil VTS

a) Configuration de l'outil

VTS vous est fourni dans le dossier « Environnement_de_developpement/Vts-WindowsNT-32bits-3.5.1 ».

Remarque : par défaut, nous fournissons une instance de VTS Windows, cela ne fonctionnera donc pas avec un PC Linux. Si vous n'avez pas le choix et devez travailler sur un PC Linux, nous vous invitons à la procédure d'installation détaillée à la fin de cette section.

Il doit impérativement être à cet emplacement pour fonctionner correctement, tel que décrit dans l'arborescence du dossier « Environnement_de_developpement » dans la section d'installation de l'environnement de développement. En effet, les codes du projet Java viennent écrire dans un sous dossier du dossier « Vts-WindowsNT-32bits-3.5.1 » et si vous le déplacez, cela ne fonctionnera plus.

Voici la structure du dossier VTS et son contenu : dossier "Vts-WindowsNT-32bits-3.5.1" contenant VTS v3.5.1, à l'intérieur vous trouverez :

- Différents dossiers permettant le fonctionnement de VTS que nous ne détaillons pas ici
- Dossier "Data" : dossier "BESupaero" déjà configuré :
 - Projet VTS préconfiguré : fichier "Mission_VTS_File.vts"
 - Dossier "Data" pour les outputs CIC du code Java (outputs de la classe **VTSTools**) :
 - Fichiers AEM pour les lois d'attitudes (quaternions)
 - Fichiers OEM pour les éphémérides de trajectoire (position/vitesse)
 - Fichiers MEM pour les événements orbitaux
 - Fichiers POI pour les sites d'intérêt (villes)
 - Dossier "Model" pour configurer la visualisation 3D du satellite

Vous n'avez pas besoin de modifier quoique ce soit pour pouvoir utiliser VTS, tout est fait automatiquement par les utilitaires Java fournis avec le projet. Nous vous demandons donc de ne surtout pas modifier les fichiers VTS.

b) Utilisation de l'outil

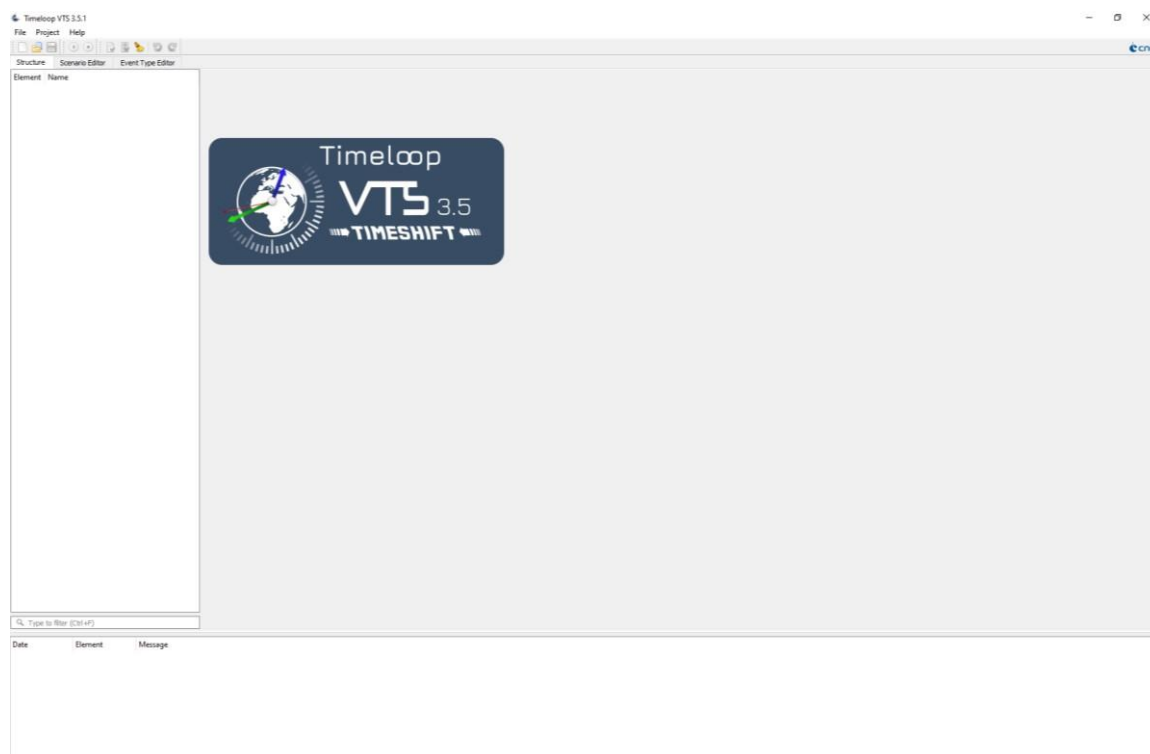
1. Pour utiliser VTS et visualiser les dernières éphémérides de votre satellite, commencez par exécuter **SimpleMissionMain** (loi nadir simple) ou **CompleteMissionMain** (votre propre mission). Les sorties des codes Java vont automatiquement être écrites dans « Vts-WindowsNT-32bits-3.5.1/Data/BESupaero/Data » et vous devriez ainsi voir les six fichiers de ce dossier s'actualiser.
2. Lancez ensuite l'exécutable « startVTS.exe » dans le dossier « Vts-WindowsNT-32bits-3.5.1 ».

Environnement de developpement > Vts-WindowsNT-32bits-3.5.1

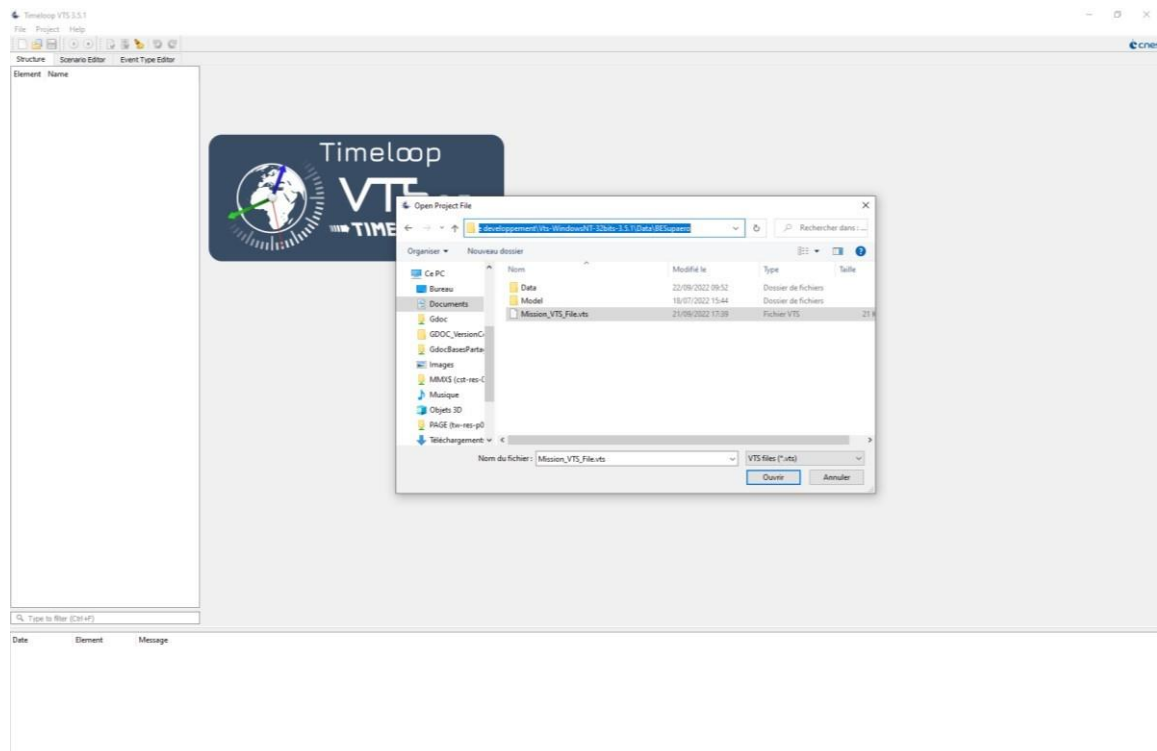
Rechercher dans : Vts-WindowsNT-32bits-3.5.1

Nom	Modifié le	Type	Taille
Apps	23/06/2022 15:24	Dossier de fichiers	
Data	08/09/2022 17:26	Dossier de fichiers	
Docs	23/06/2022 15:24	Dossier de fichiers	
Generators	23/06/2022 15:24	Dossier de fichiers	
ChangeLog.txt	20/01/2021 19:22	Document texte	39 Ko
LICENSE-EN.txt	20/01/2021 19:22	Document texte	4 Ko
LICENSE-FR.txt	20/01/2021 19:22	Document texte	5 Ko
startVTS.exe	20/01/2021 19:22	Application	65 Ko

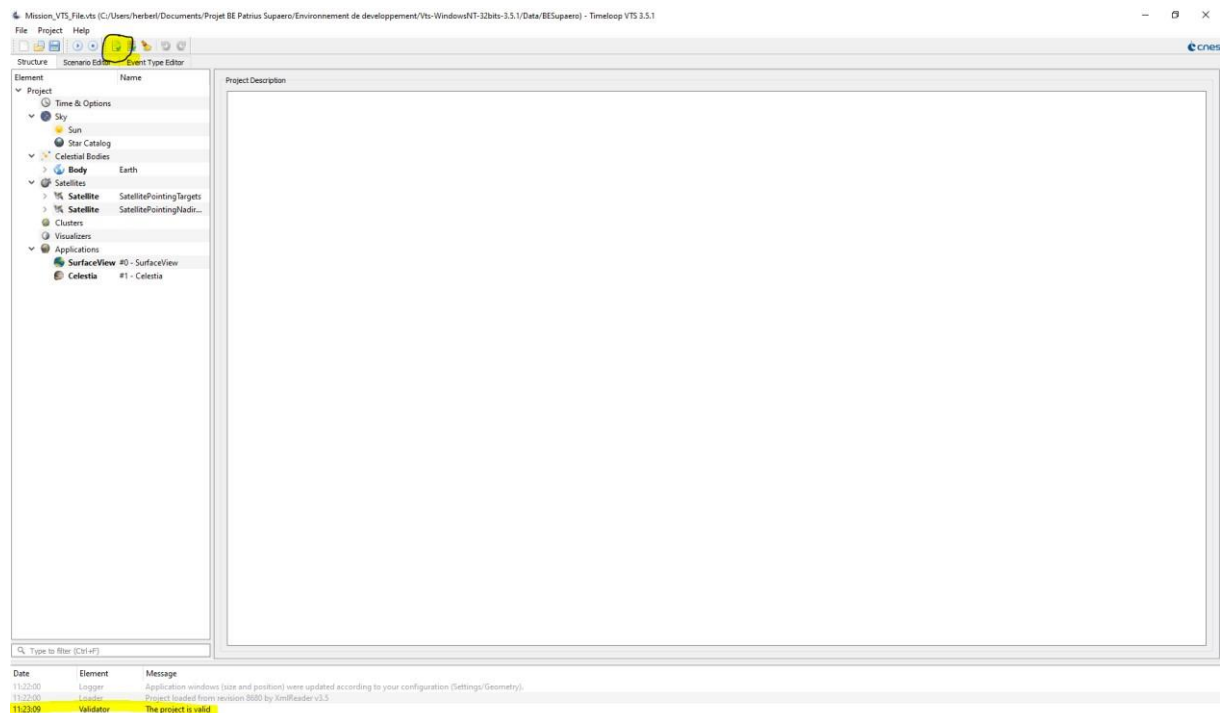
3. Puis vous devriez avoir la fenêtre suivante qui s'affiche.



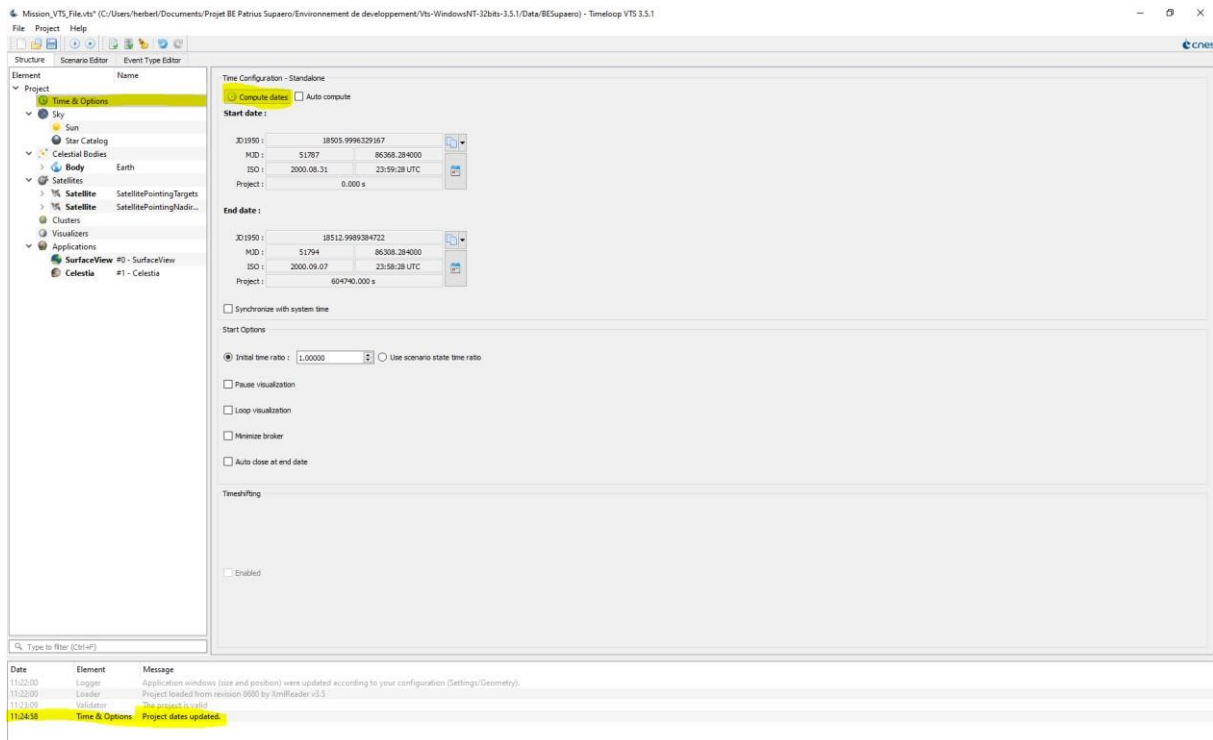
4. Dans « File > Open Project », sélectionnez alors le fichier « Vts-WindowsNT-32bits3.5.1/Data/BESupaero/Mission_VTS_File.vts ».



5. Vous devriez ensuite obtenir ceci (voir la capture d'écran ci-dessous). Cliquez sur « Check Project » pour vérifier que le projet est valide, l'output sera affiché en bas.



6. Si le projet n'est pas valide, demandez de l'aide aux intervenants. Il se peut sinon que vous deviez actualiser les dates de la simulation avant de lancer le projet en suivant les indications de la capture d'écran ci-dessous :



Note : configuration du projet VTS

Ne modifiez surtout pas la configuration VTS

Quelques mots à propos du projet VTS. Sa configuration vous est affichée dans la partie gauche de l'IHM, sous l'onglet « Project ». Ici vous pouvez voir apparaître toute l'arborescence de la configuration du projet, sous forme d'arbre avec des objets. Tout ceci est écrit dans le fichier « Mission_VTS_File.vts » qui est en fait un dérivé du format XML.

VTS fonctionne avec un système d'objets parents et d'objets enfants.

Vous pouvez voir qu'on a un contexte physique modélisé par des objets célestes : « Sky » représente le fond céleste, avec « Sun » qui est l'objet parent de tous les autres objets du système solaire, et « Star Catalog » qui est un catalogue d'étoiles. Ensuite, l'onglet « Celestial Bodies » permet de configurer les corps célestes. Ici nous n'avons que des corps du système solaire, et en l'occurrence pour nous la Terre avec l'objet « Earth ». Sun est le parent de Earth, le référentiel de Earth dépend donc de celui de Sun. Si vous déroulez l'objet Earth, vous trouverez ses sous-objets : stations sol, sites d'intérêt, etc...

Ensuite, tous les satellites configurés dans l'onglet « Satellites » sont des objets enfants de Earth. Nous en avons implémenté deux : un qui va toujours viser nadir afin de faire apparaître le cône de visibilité (le cône de dépointage autorisé) et l'autre qui va exécuter les instructions écrites en sortie de la simulation, à savoir votre plan cinématique. Si vous déroulez le menu de chaque satellite, vous verrez apparaître leur configuration. Vous pouvez notamment voir qu'on a configuré un capteur et son champ de vue par satellite, et qu'on définit également leurs éphémérides de position/vitesse et d'attitude via les fichiers présents dans le répertoire data.

7. Si le projet est bien valide, vous pouvez finalement lancer la simulation pour visualiser vos sorties avec la commande « Run » comme indiqué sur la capture d'écran suivante :



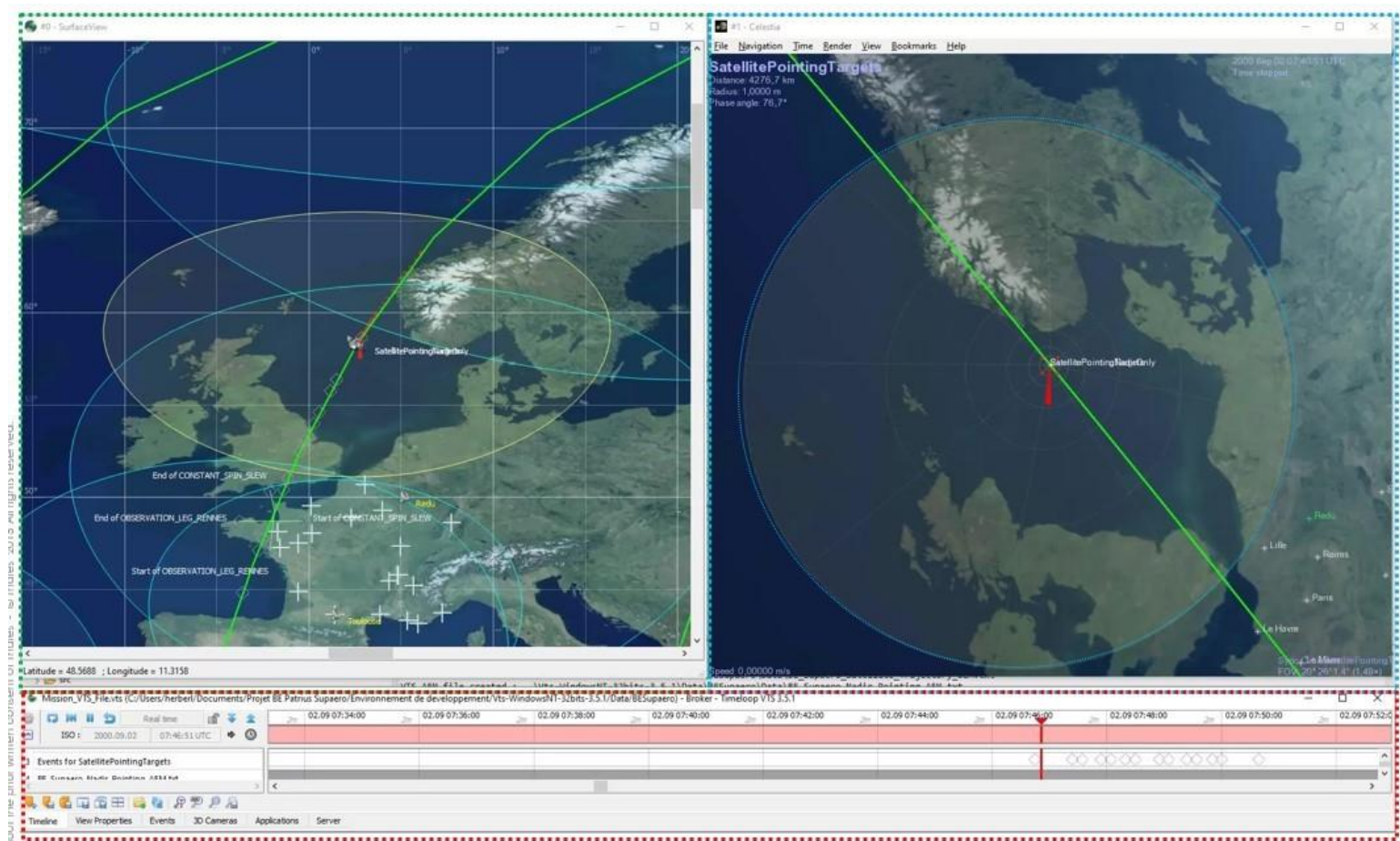
Note : Pour afficher l'aide, cliquez sur « Help » comme indiqué sur la capture d'écran suivante :



Une fois la simulation lancée, vous devriez voir trois fenêtres interactives s'ouvrir :

- La fenêtre **Broker VTS**, qui permet d'afficher les chronogrammes d'évènements, choisir les vues caméra et contrôler l'écoulement du temps
- La fenêtre **Celestia**, qui affiche en 3D la mission, où vous pouvez suivre un objet ou vous déplacer (déplacer la caméra) pour visualiser ce que vous voulez
- La fenêtre **SurfaceView**, qui affiche une projection sur un planisphère de l'orbite et de l'empreinte au sol des capteurs

Les contrôles de la simulation sont très intuitifs, n'hésitez pas à jouer avec la simulation pour comprendre comment elle fonctionne. Vous aurez l'occasion de manipuler VTS en séance, n'hésitez pas à utiliser l'aide VTS ou à demander de l'aide à un intervenant.



c) Installation de VTS pour Linux

Si vous avez un PC Linux, vous devrez utiliser la version Linux de VTS en version 3.5.1.

Si vous avez la moindre difficulté lors de cette procédure d'installation, n'hésitez pas à nous contacter par mail !

Voici la procédure d'installation :

1. Téléchargez le client VTS en version « VTS 3.5.1 "Timeshift" » sur ce site : <https://timeloop.fr/vts/download/>. Il vous faudra remplir un petit formulaire pour accéder au lien de téléchargement.
2. Vous devriez obtenir un fichier tar.gz nommé « Vts-Linux-64bits-3.5.1.tar.gz », il vous faut extraire l'archive et la placer dans le dossier « Environnement_de_developpement » au même niveau que « BEProgrammationMission » et que « Vts-WindowsNT-32bits-3.5.1 » qui vous est fourni. Votre client VTS local est installé, maintenant il faut le configurer pour que vous puissiez l'utiliser dans le cadre du BE.
3. Dans le répertoire « Data » de ce dossier qui devrait s'appeler « Vts-Linux-64bits-3.5.1 », il faut ensuite ajouter le dossier « BESupaero » situé ici : « Vts-WindowsNT-32bits-3.5.1/Data/BESupaero » et tout son contenu. Une fois cela fait, tout est en place côté VTS.

4. La dernière étape consiste à brancher les sorties VTS du projet Java sur les entrées du projet VTS. Pour cela, il faudra mettre à jour la variable *PATH_VTS_DIRECTORY* de la classe ***ConstantsBE*** du projet Java de manière à lui indiquer le chemin relatif vers le dossier des entrées du projet VTS. Ce chemin est par défaut défini comme étant «../Vts-WindowsNT-32bits-3.5.1/Data/BESupaero/Data », nous vous laissons le remplacer par le bon chemin dans le client Linux (quasi identique mais dans le dossier du client VTS Linux au lieu du dossier du client Windows).
5. Testez que tout fonctionne bien en exécutant la classe ***SimpleMissionMain***, vérifiez ensuite que les fichiers VTS ont bien été exportés au bon endroit puis lancez le projet VTS pour vérifier que la simulation fonctionne bien.